-1. Line 6 converts 100 into a high-precision real number and computes e100, the exponential function. The result is a very large number printed using 100-bit precision. Line 7 prints the precision of x, confirming that it is stored with 100 bits of precision. Line 8 replaces the old value of x with a new high-precision number (√3). Line 9 prints the high-precision decimal approximation of √3. Line 10 prints the next representable floating-point number greater than x. It is the smallest number larger than √3 that can be stored in the 100-bit system. Line 11 prints the difference between √3 and the next representable number. That value is the ULP (Unit in the Last Place) at √3 — the smallest spacing between numbers at that magnitude.

When the program 'functions46.sage' is executed, the output shown in Figure 3 is produced. The sixth and seventh lines display the approximate value of √3 and the next representable value greater than √3. At first glance, this result appears surprising because both numbers are printed identically. However, they are not equal internally. The difference becomes evident in the following line of the output, where the subtraction of the two values yields a nonzero result — a very small fraction representing the spacing between adjacent floating-point numbers. This example illustrates the care that SageMath and the MPFR library take in preserving numerical precision. Even when two values appear identical when printed, the system maintains their exact internal distinction at the level of machine precision.

```
-1.4142135623730950488016887242
1.4142135623730950488016887242
-1
2.6881171418161354484126255516e43
100
1.7320508075688772935274463415
1.7320508075688772935274463415
1.5777218104420236108234571306e-30
```

Figure 3: Output of a few more MPFR functions

What we have discussed here is only the tip of the iceberg. MPFR is a powerful library with extensive capabilities, and it is well worth exploring further. We now turn to another important topic in number theory: primality testing.

In the previous articles in this series, where we discussed cybersecurity related topics, we performed prime factorization and primality testing. However, we did not specify which primality testing algorithm was used. Broadly, primality testing algorithms are classified into two types: deterministic and probabilistic. Deterministic algorithms always produce a correct result, but they are often slower than many probabilistic algorithms. Probabilistic algorithms, on the other hand, may occasionally produce an incorrect result. A probabilistic primality test will never report a prime number as composite. However, it may classify a composite number as a probable prime.

Let us now examine some important deterministic primality testing algorithms. Sieve-based methods form a family of algorithms that are primarily used for generating all prime numbers up to a given limit, rather than testing the primality of a single very large integer. Among these, the most famous is the Sieve of Eratosthenes, named after the Greek mathematician Eratosthenes, who is also credited with the first known measurement of the Earth's circumference.

The sieve works by systematically eliminating composite numbers. First, list all integers from 2 up to the desired limit. Circle the first number (2) and cross out all its multiples. Move to the next uncrossed number (3), circle it, and again eliminate its multiples. This process continues, advancing to the next uncrossed number each time. Once all multiples of primes up to √n have been removed, the numbers that remain uncrossed are precisely the prime numbers.

Modern sieve-based techniques, such as segmented sieve, are used internally in SageMath through high-performance numerical libraries such as GMP and PARI/GP. The following code generates and prints all prime numbers less than 1000 by leveraging these optimised libraries.

```
n = primes(1000)
for i in n:
    print(i)
```

Although it is not used in SageMath, it is important to mention another landmark deterministic primality testing algorithm: the AKS primality test, developed by Manindra Agrawal, Neeraj Kayal, and Nitin Saxena at the Indian Institute of Technology Kanpur. Their groundbreaking paper, 'PRIMES is in P', proved for the first time that primality testing can be done in deterministic polynomial time. The work was widely celebrated and received both the Gödel Prize and the Fulkerson Prize in 2006, recognising its profound impact on theoretical computer science and discrete mathematics. It remains one of the most significant achievements by Indian computer scientists in modern times.

Let us now discuss some probabilistic primality testing algorithms. Important members of this class include the Fermat test, the Solovay–Strassen test, the Miller–Rabin test, and the Baillie–PSW test. The Fermat and Solovay–Strassen tests are primarily of theoretical and academic interest, whereas the Miller–Rabin and Baillie–PSW tests are widely used in practical applications. A full technical treatment of these algorithms is beyond the scope of this series; however, it is useful to understand how the confidence level of probabilistic tests can be amplified. Most probabilistic primality tests guarantee that the probability of incorrectly declaring a composite number to be prime is at most 1/4 in